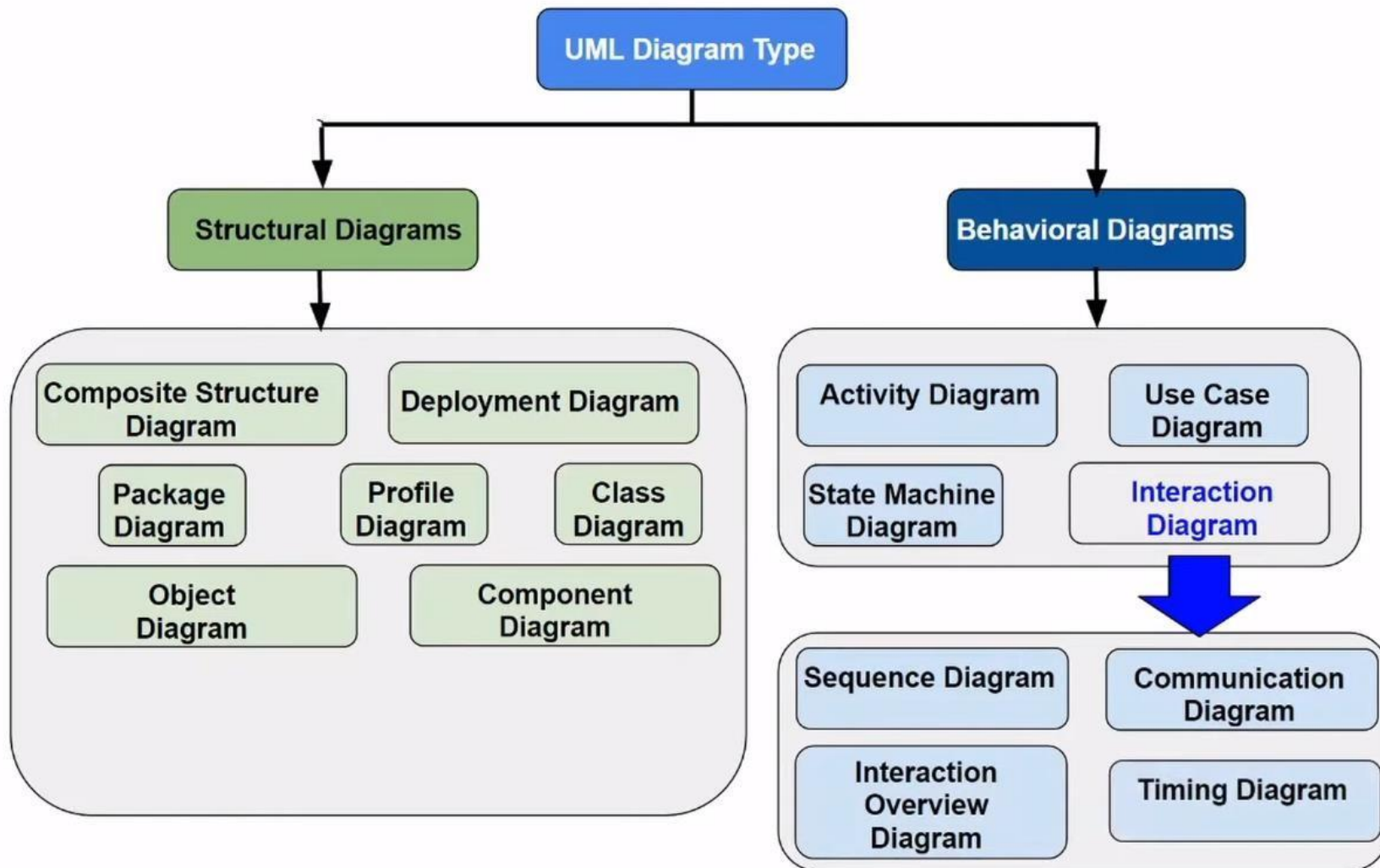


**UNIFIED
MODELING
LANGUAGE™**





UML Standard Diagrams

Structural Diagrams

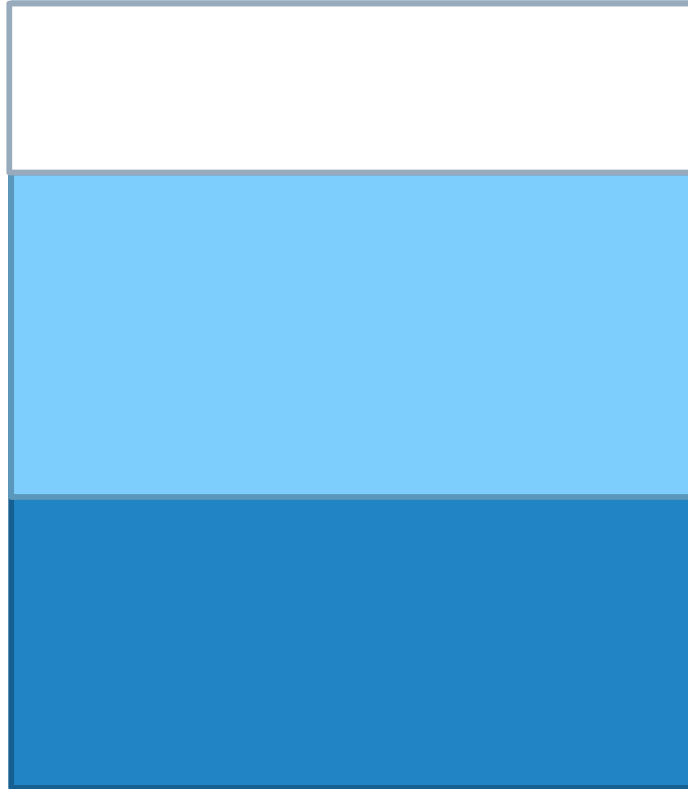
- **Behavioural Diagrams**

Structural Diagrams

- Class diagram
- Object diagram
- Component diagram
- Deployment diagram

Class Diagram

Class Block



Vital components of a Class Diagram

Upper Section: The upper section encompasses the name of the class. A class is a representation of similar objects that shares the same relationships, attributes, operations, and semantics. Some of the following rules that should be taken into account while representing a class are given below:

- Capitalize the initial letter of the class name.
- Place the class name in the center of the upper section.
- A class name must be written in bold format.
- The name of the abstract class should be written in italics format.

Vital components of a Class Diagram

Middle Section: The middle section constitutes the attributes, which describe the quality of the class. The attributes have the following characteristics:

- The attributes are written along with its visibility factors, which are public (+), private (-), protected (#), and package (~).
- The accessibility of an attribute class is illustrated by the visibility factors.
- A meaningful name should be assigned to the attribute, which will explain its usage inside the class.

Lower Section: The lower section contain methods or operations. The methods are represented in the form of a list, where each method is written in a single line. It demonstrates how a class interacts with data.

Relationships between classes in UML

Association



Inheritance



Realisation



Dependency



Aggregation

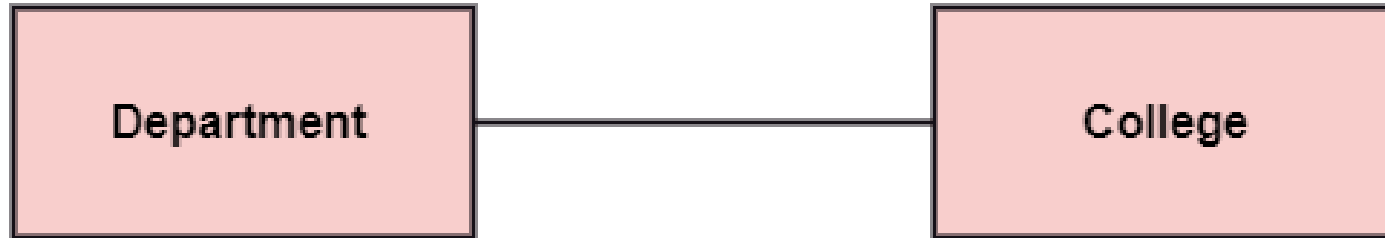


Composition



Association

- Association: It describes a static or physical connection between two or more objects. It depicts how many objects are there in the relationship.
- For example, a department is associated with the college.



Association



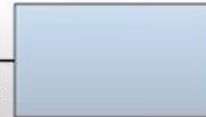
One to One

1



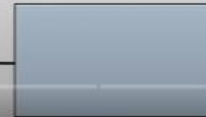
One to Many

1..*

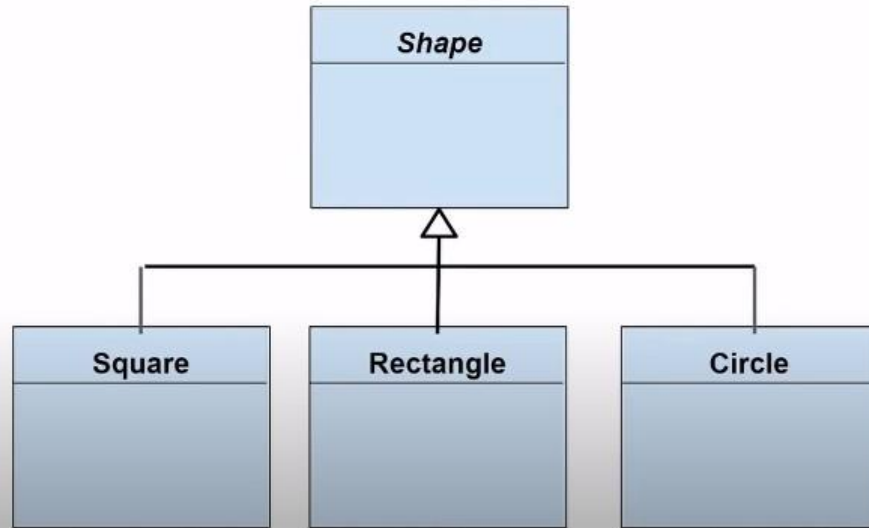


Many to Many

*



Inheritance



Aggregation

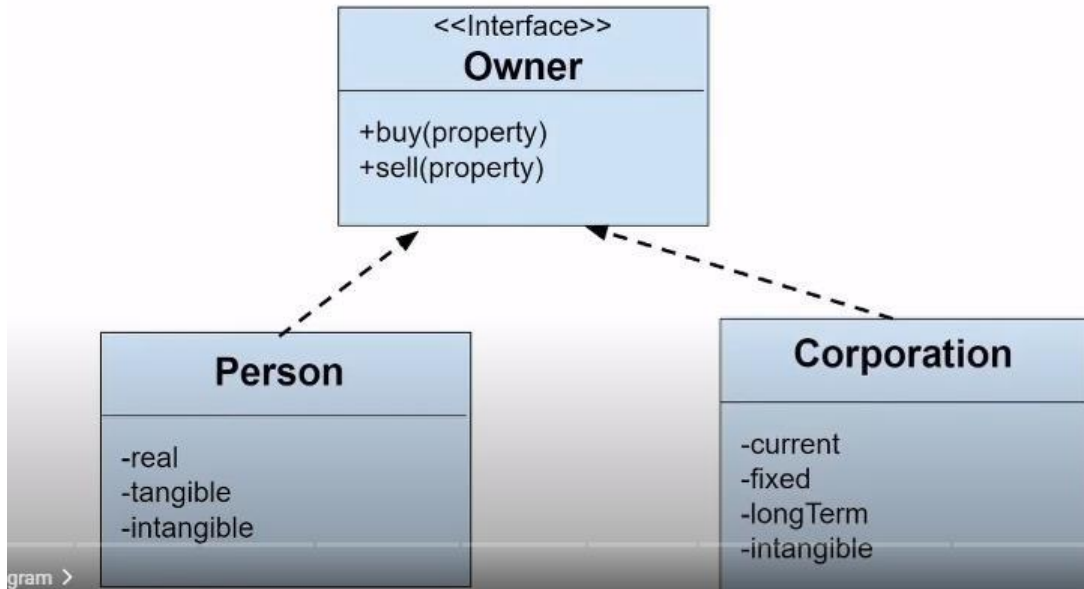
- Aggregation: An aggregation is a subset of association, which represents has a relationship. It is more specific than association. It defines a part-whole or part-of relationship. In this kind of relationship, the child class can exist independently of its parent class.



- Composition: The composition is a subset of aggregation. It portrays the dependency between the parent and its child, which means if one part is deleted, then the other part also gets discarded. It represents a whole-part relationship.



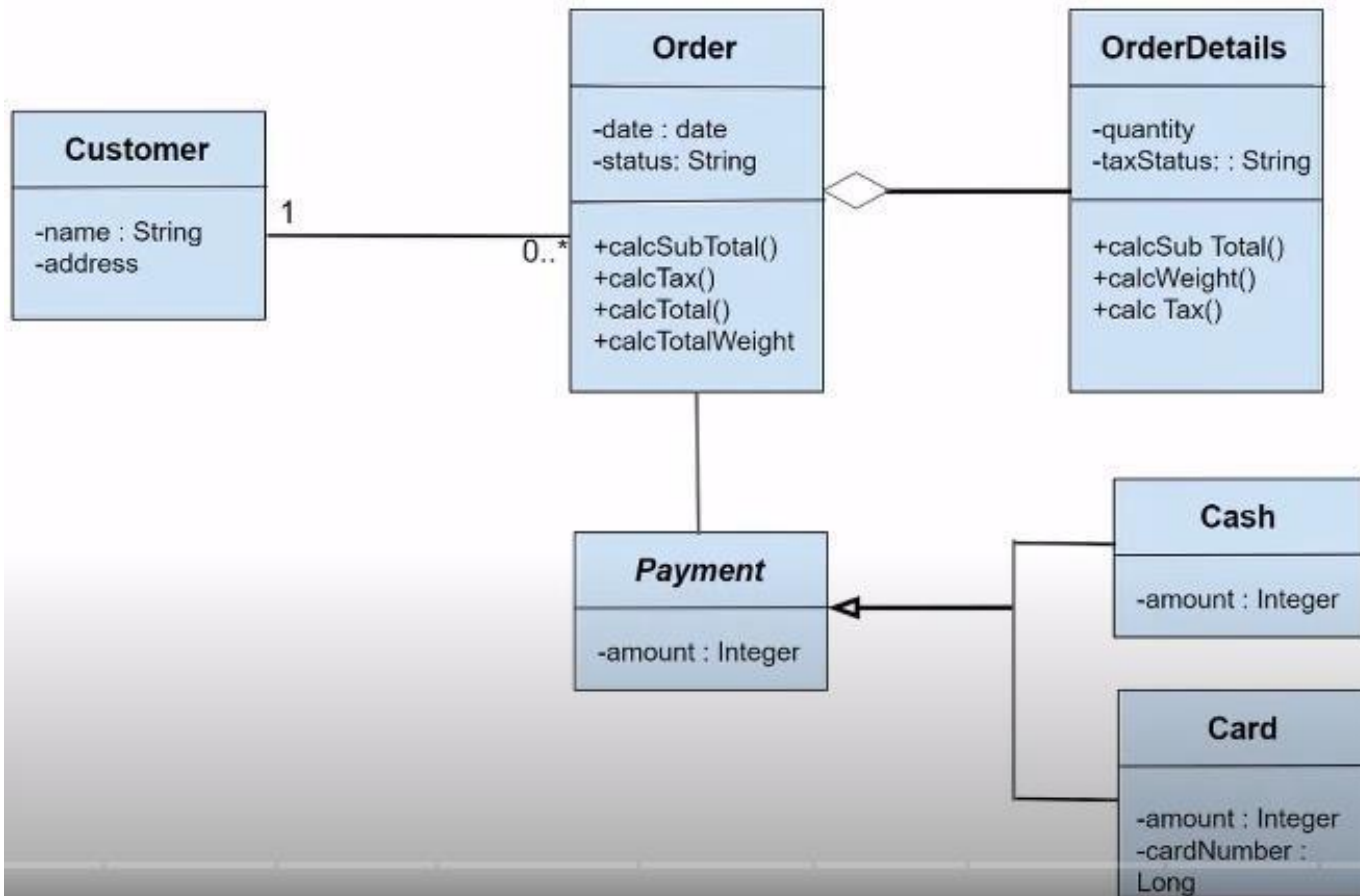
Realization



Example

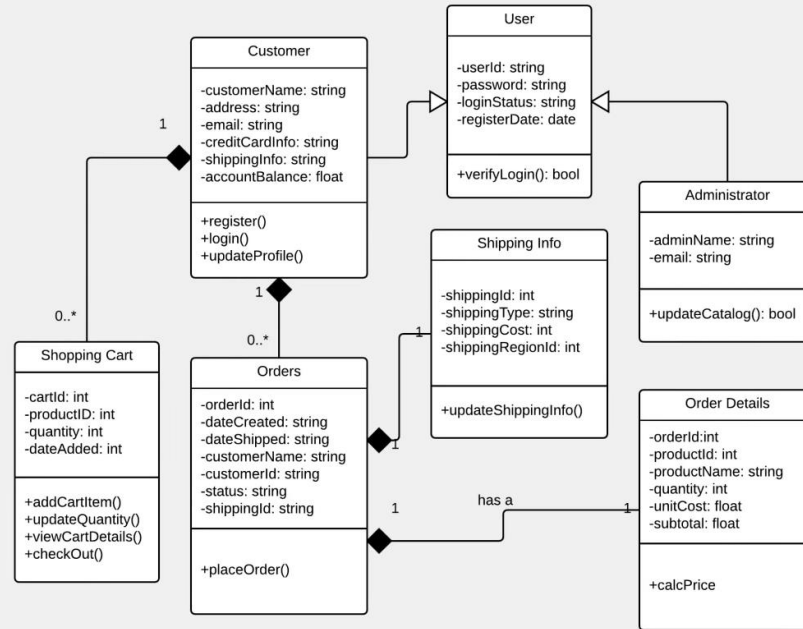
- Order System of an application





Class diagram - Static view of an application

UML CLASS DIAGRAMS

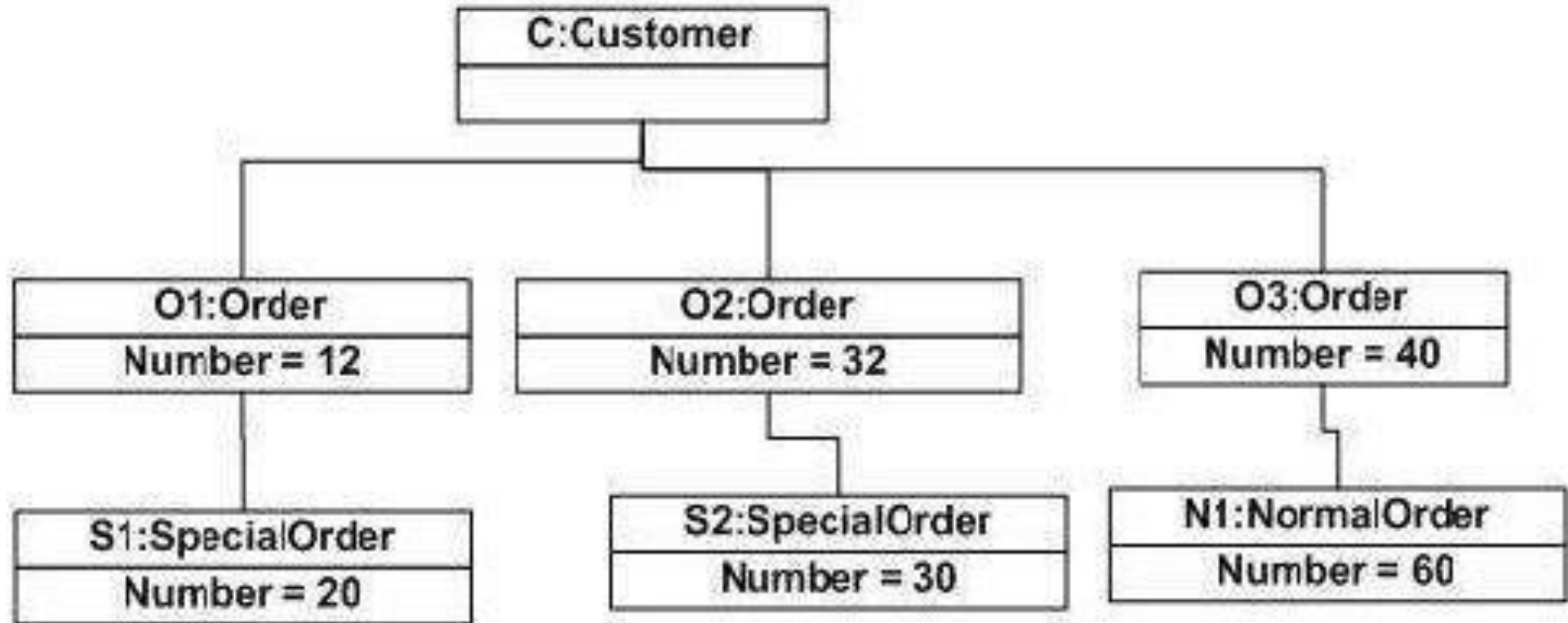


Usage of Class diagrams

- To describe the static view of a system.
- To show the collaboration among every instance in the static view.
- To describe the functionalities performed by the system.
- To construct the software application using object-oriented languages.

Object Diagrams - instance of a class diagram

Object diagram of an order management system



Component Diagram

- It models the physical view of a system such as executables, files, libraries, etc. that resides within the node.

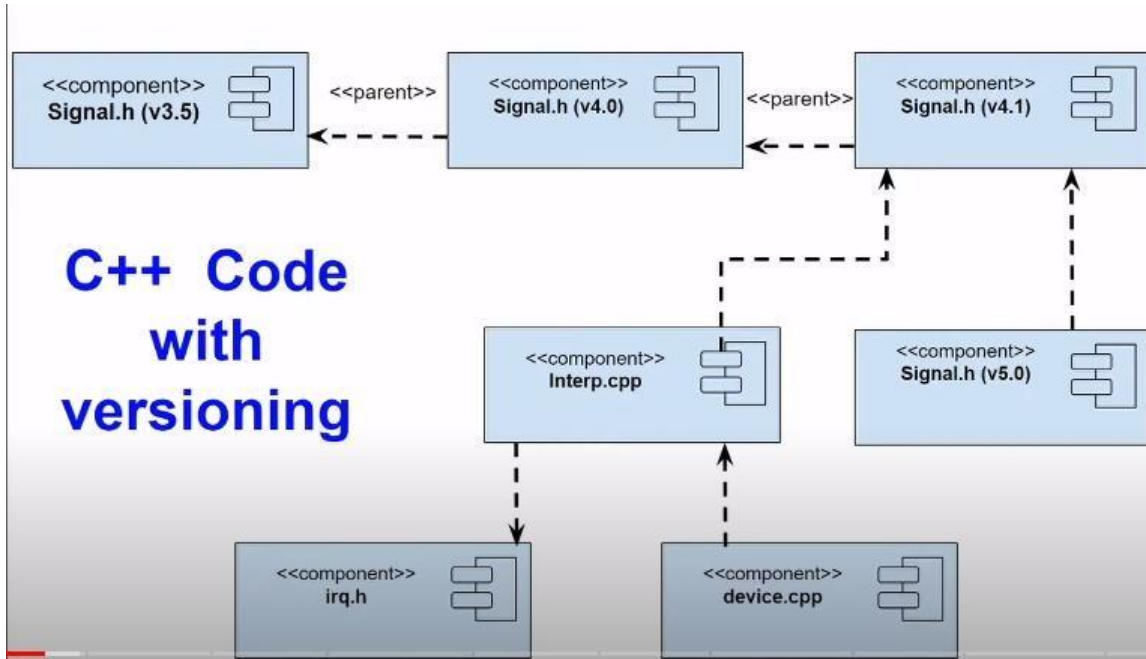
Following are some artifacts that are needed to be identified before drawing a component diagram:

- What files are used inside the system?
- What is the application of relevant libraries and artifacts?
- What is the relationship between the artifacts?

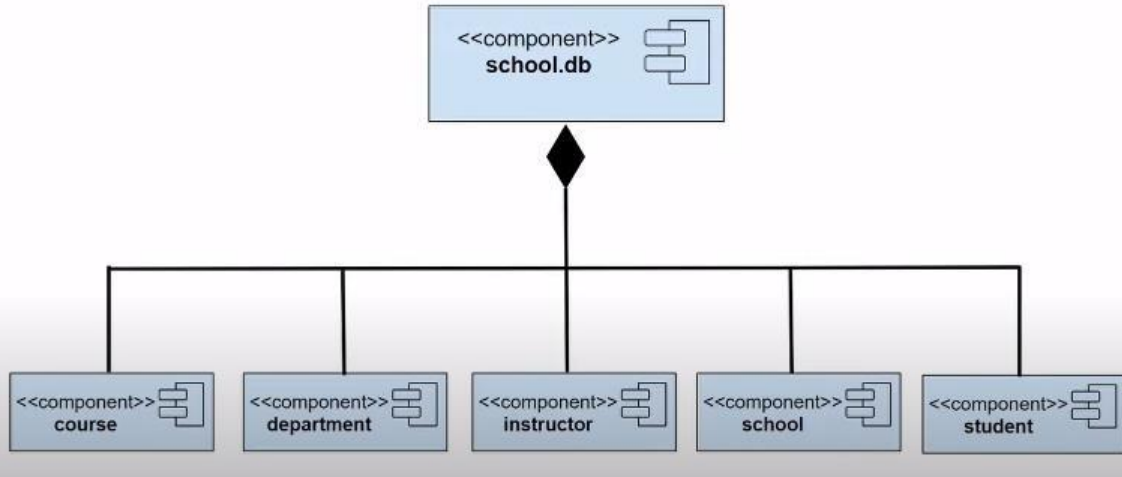
The component diagram can be used for the followings:

- To model the components of the system.
- To model the schemas of a database.
- To model the applications of an application.
- To model the system's source code.

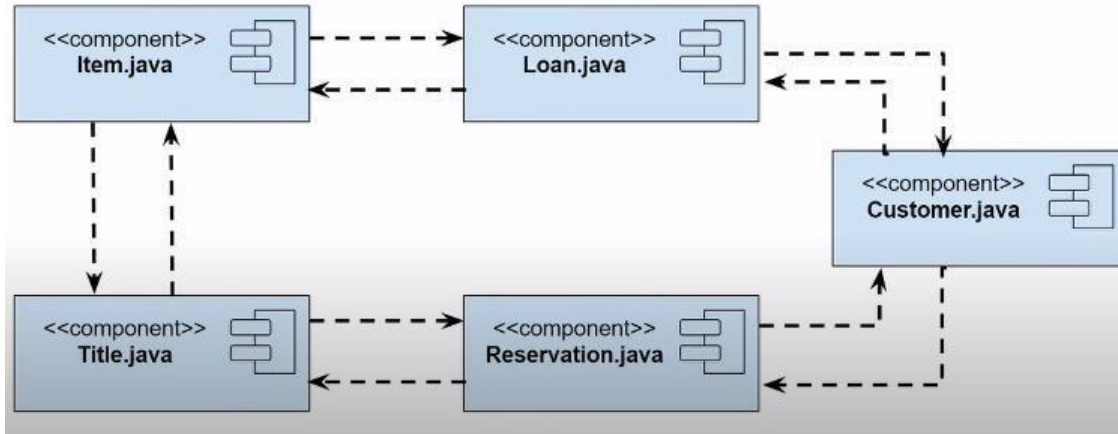
C++ Code with versioning



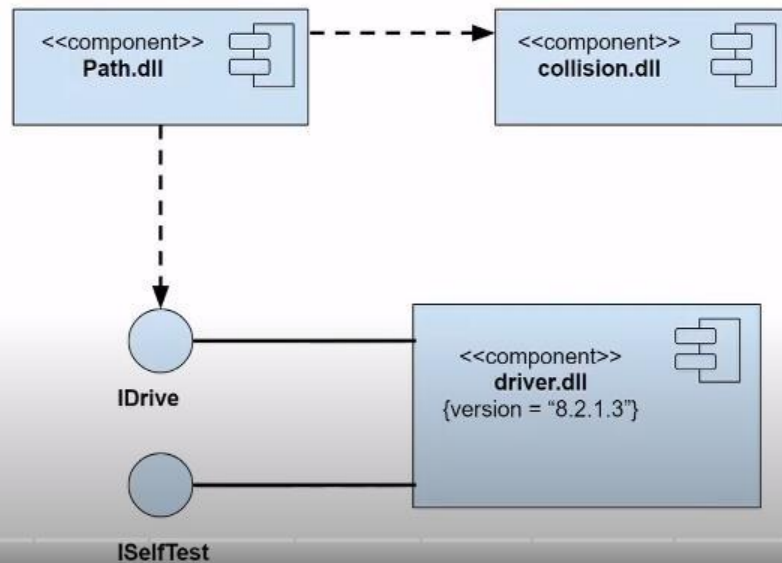
Modeling a Physical Database



Java Source Code



Modeling an Executable Release



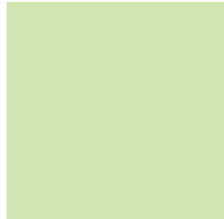
Component Diagrams - Notations



Component
symbol



Node symbol



Package
symbol

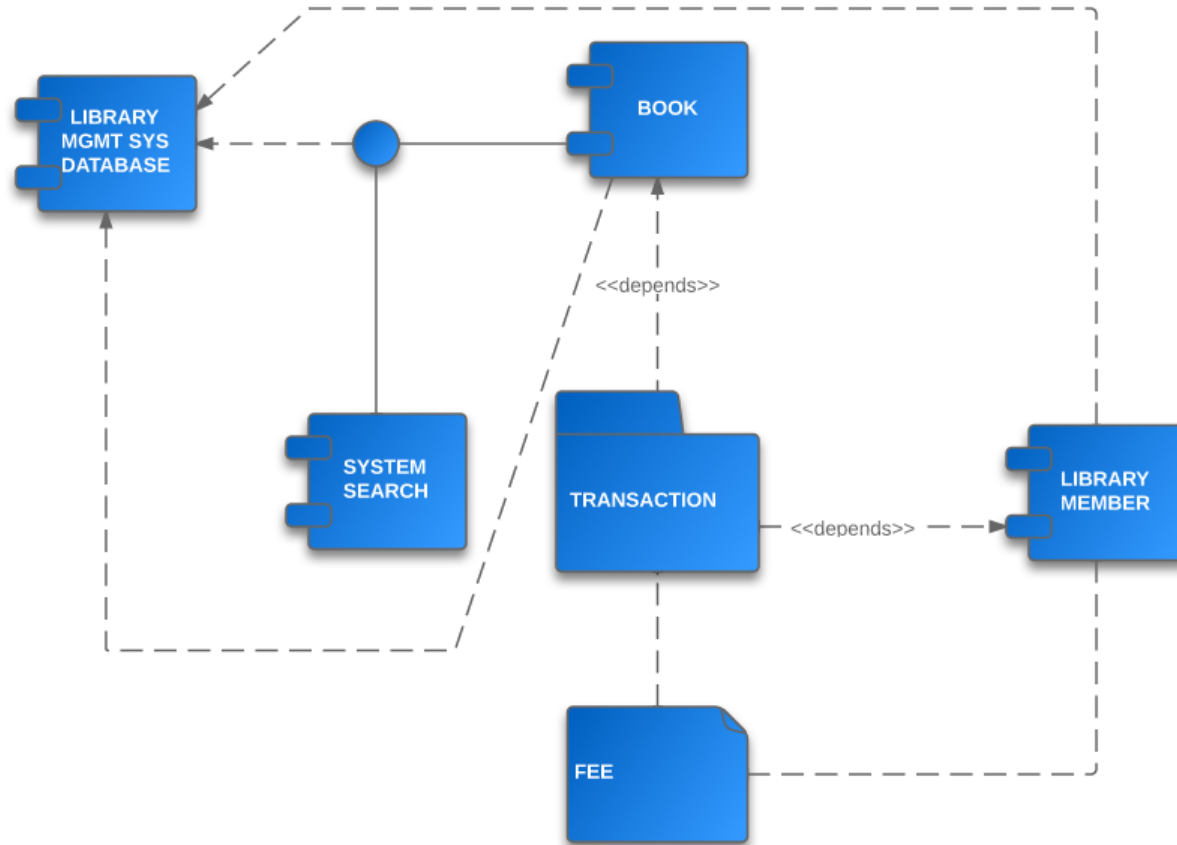


Note symbol



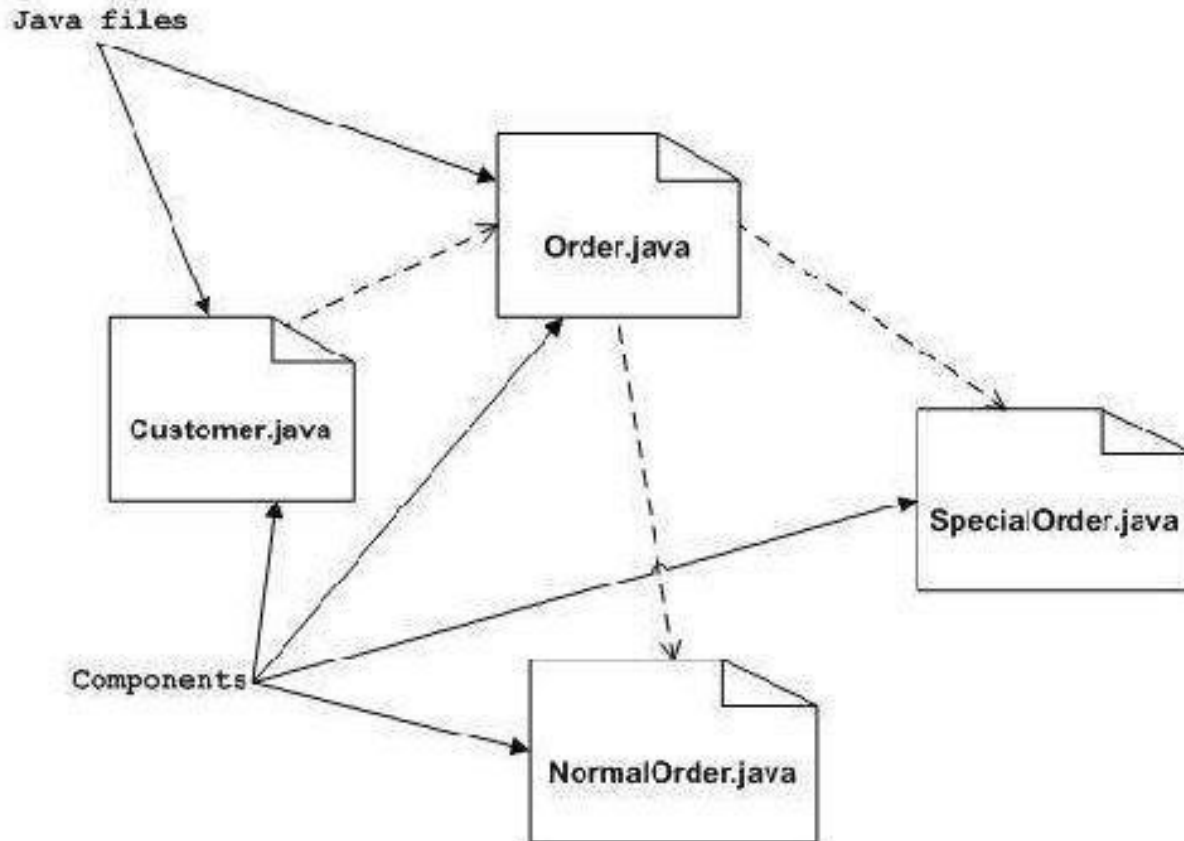
Dependency
symbol

Component Diagrams - model the physical aspects of a system



Component Diagrams - model the physical aspects of a system

Component diagram of an order management system



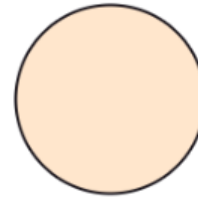
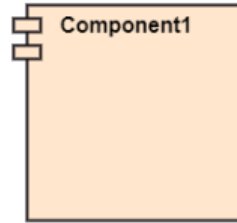
Deployment Diagrams-

to visualize the topology of the physical components of a system, where the software components are deployed

Symbol and notation of Deployment diagram

The deployment diagram consist of the following notations:

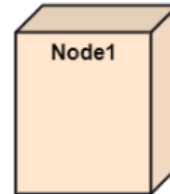
1. A component
2. An artifact
3. An interface
4. A node



Interface1



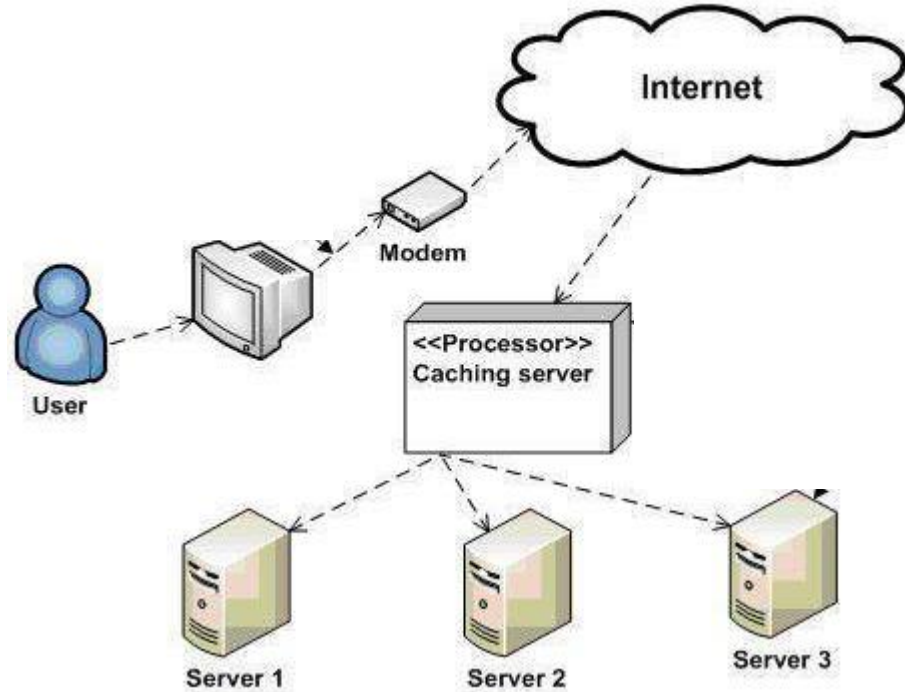
Artifact1



Node1

Deployment Diagrams-

to visualize the topology of the physical components of a system, where the software components are deployed



When to use a Deployment Diagram?

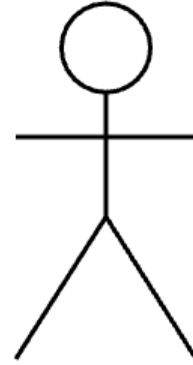
- To model the network and hardware topology of a system.
- To model the distributed networks and systems.
- Implement forwarding and reverse engineering processes.
- To model the hardware details for a client/server system.
- For modeling the embedded system.

Behavioral Diagrams - Behaviour of the system when it is running/operating

- Use case diagram
- Sequence diagram
- Collaboration diagram
- State chart diagram
- Activity diagram

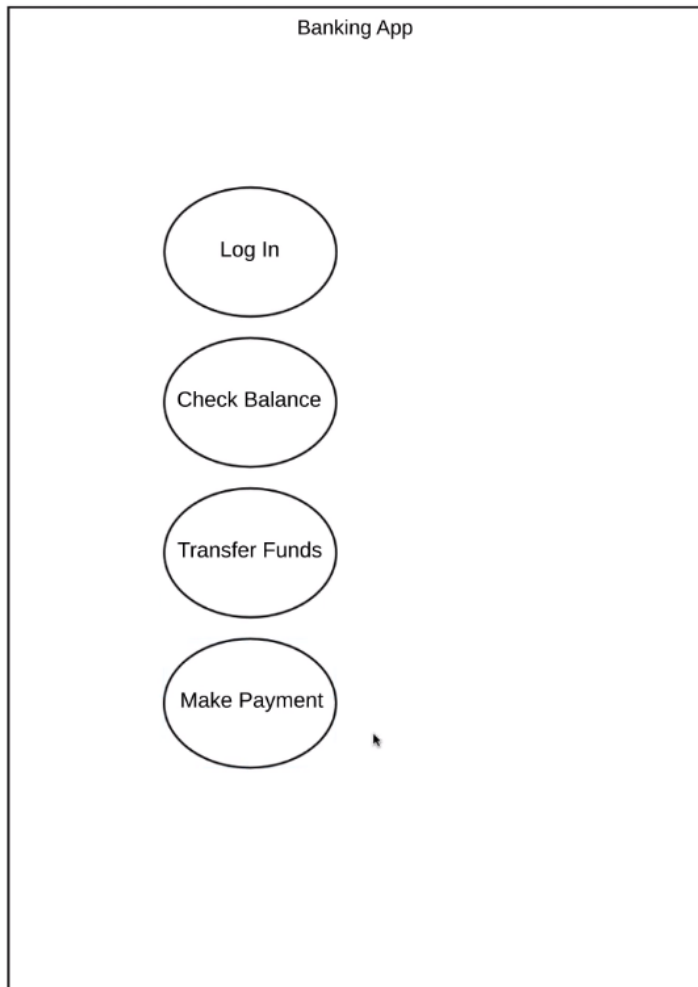
Use Case Actors

- **Primary Actors**
 - Initiate the use of the system
- **Secondary Actors**
 - Reactionary
- **Actor can be**
 - People
 - Organization
 - External Device

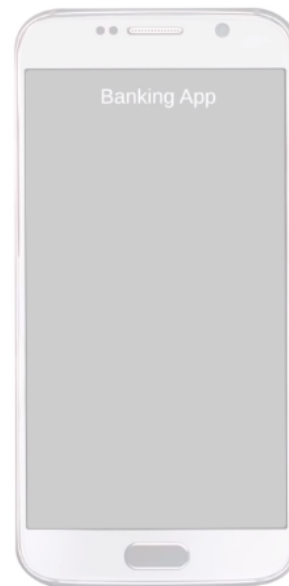




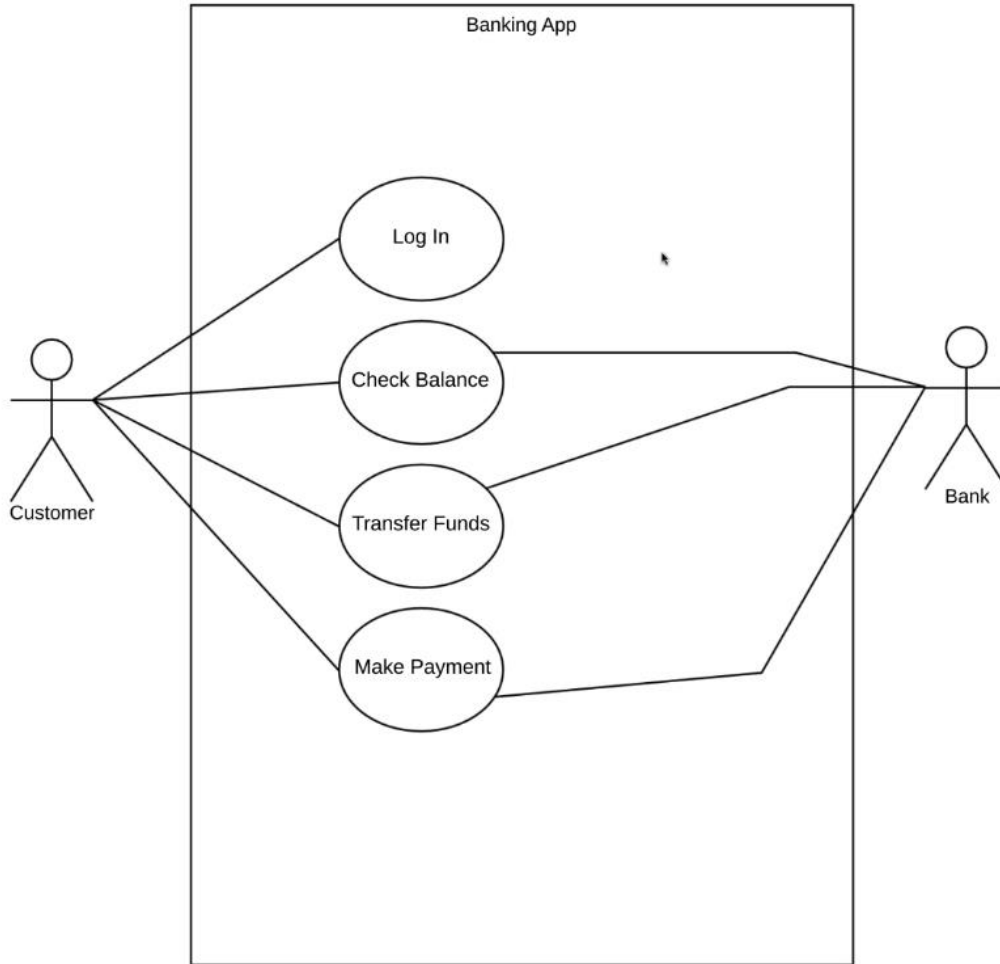
Customer



Bank



Log In
Check Balance
Transfer Funds
Make Payment



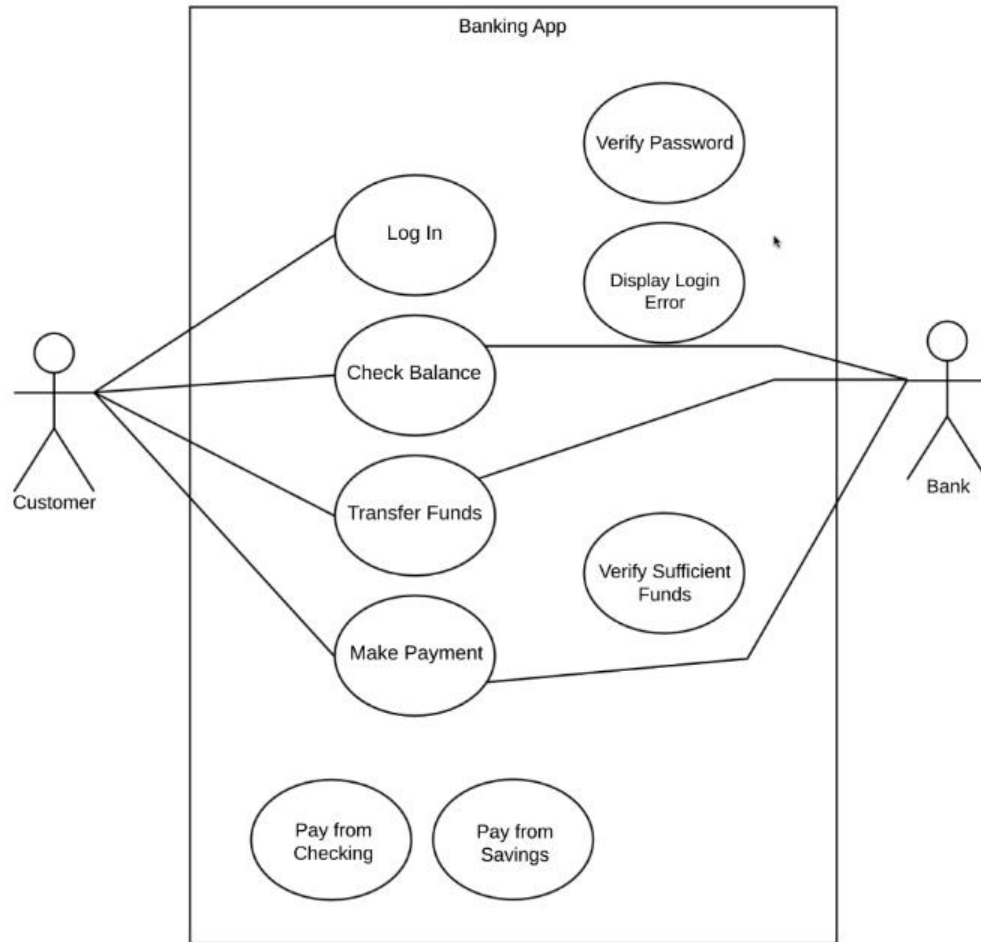
Relationships

Association

Include

Extend

Generalization



Relationships

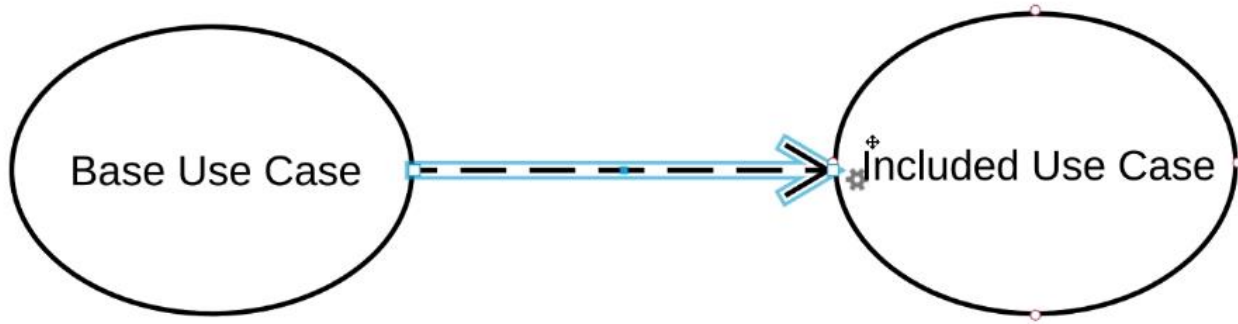
Association

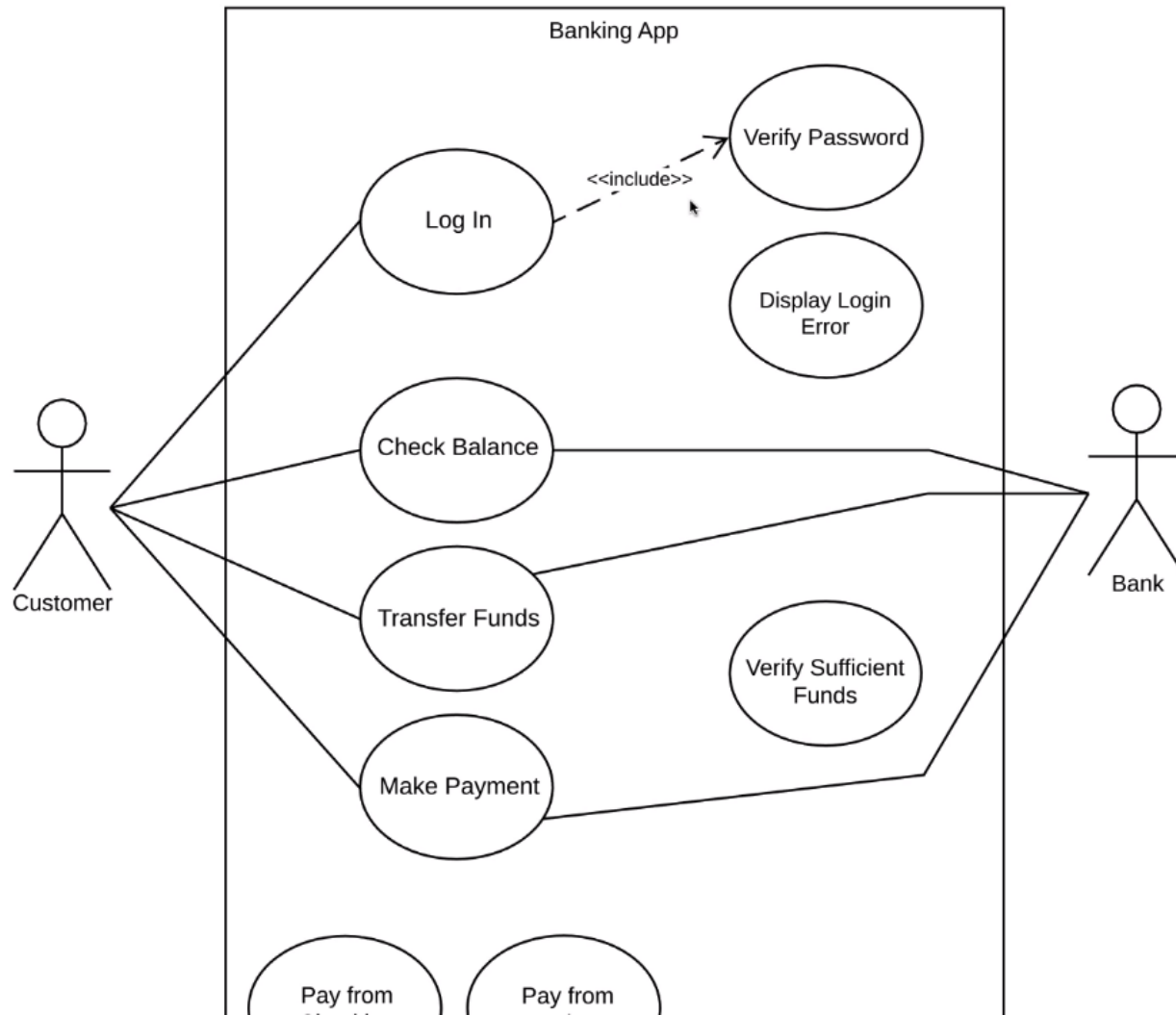
Include

Extend

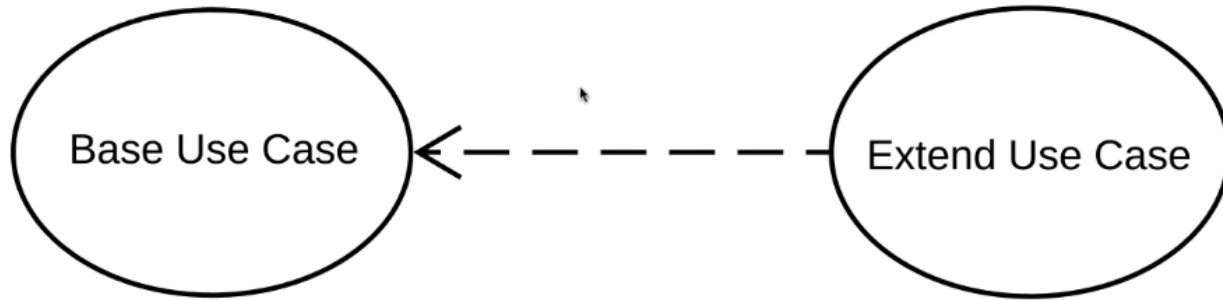
Generalization

Include

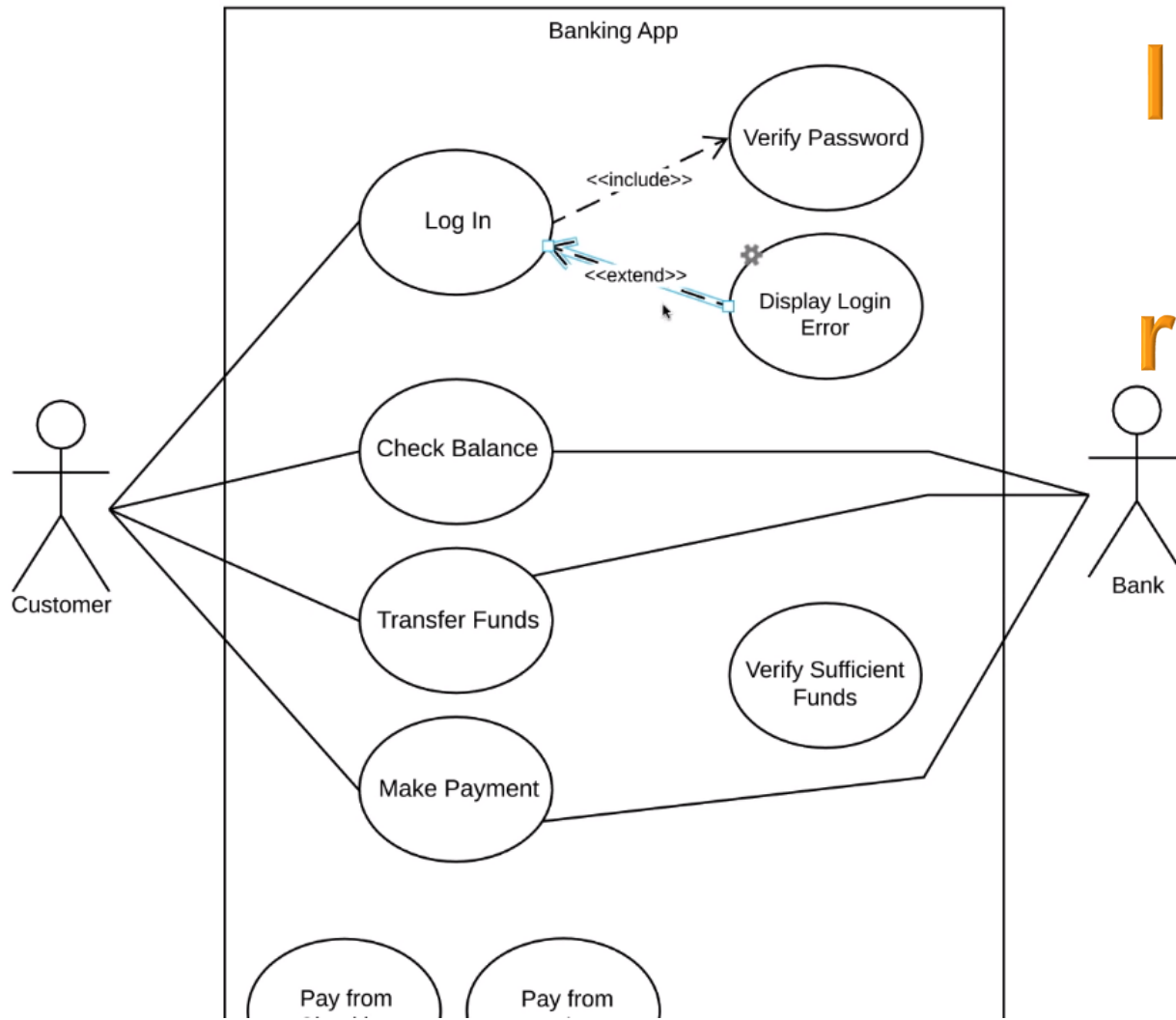


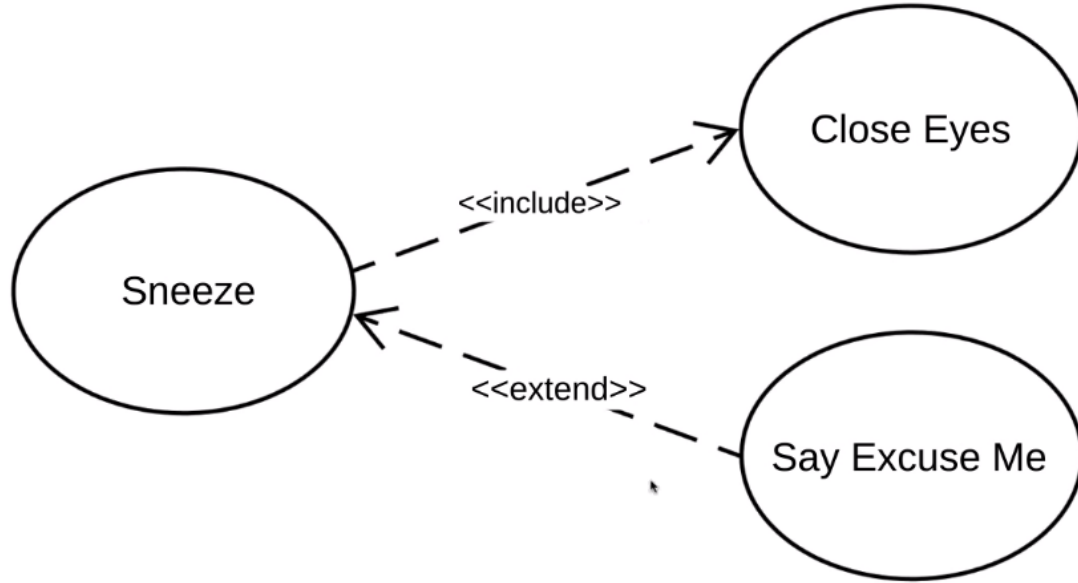


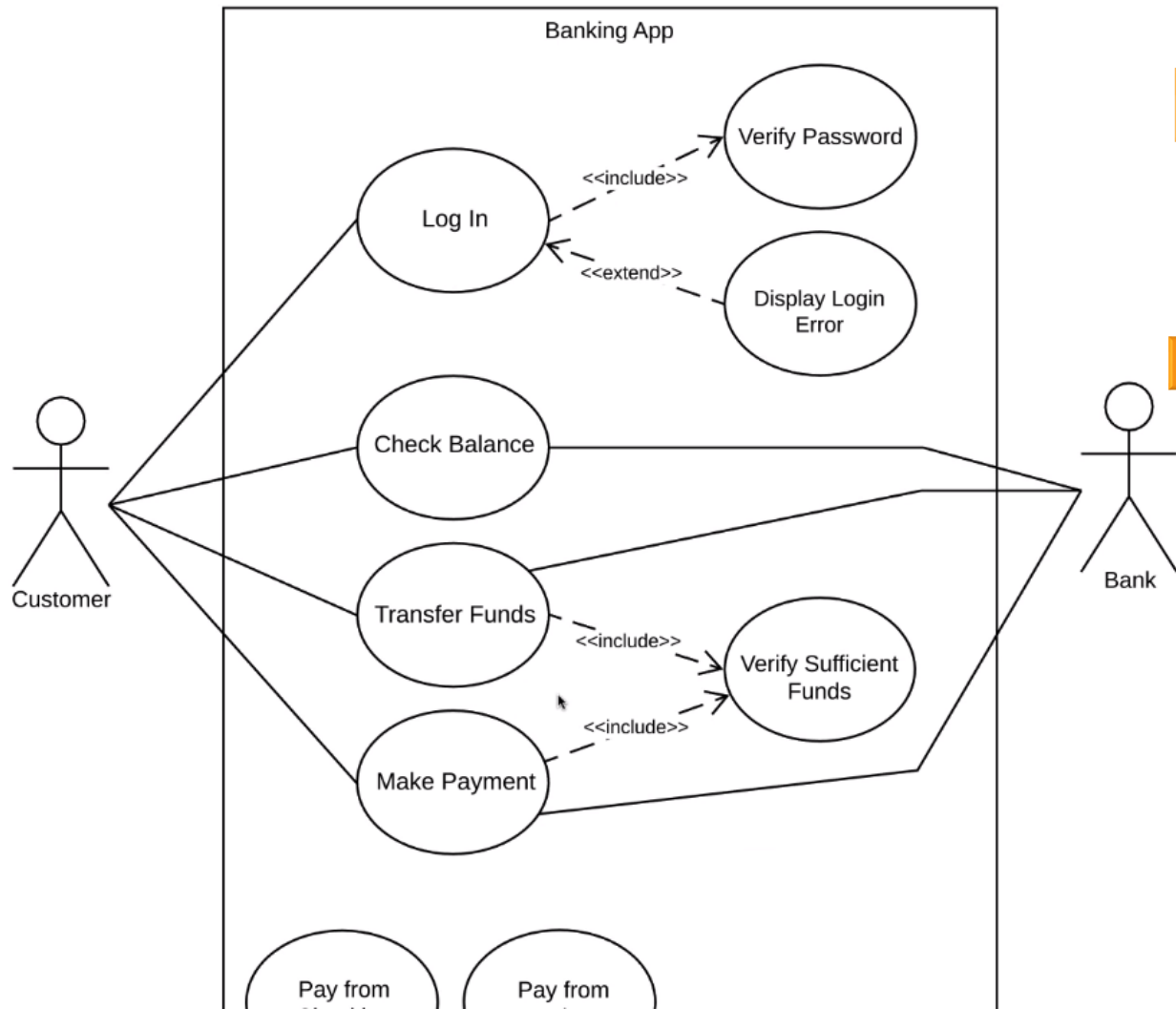
Extend



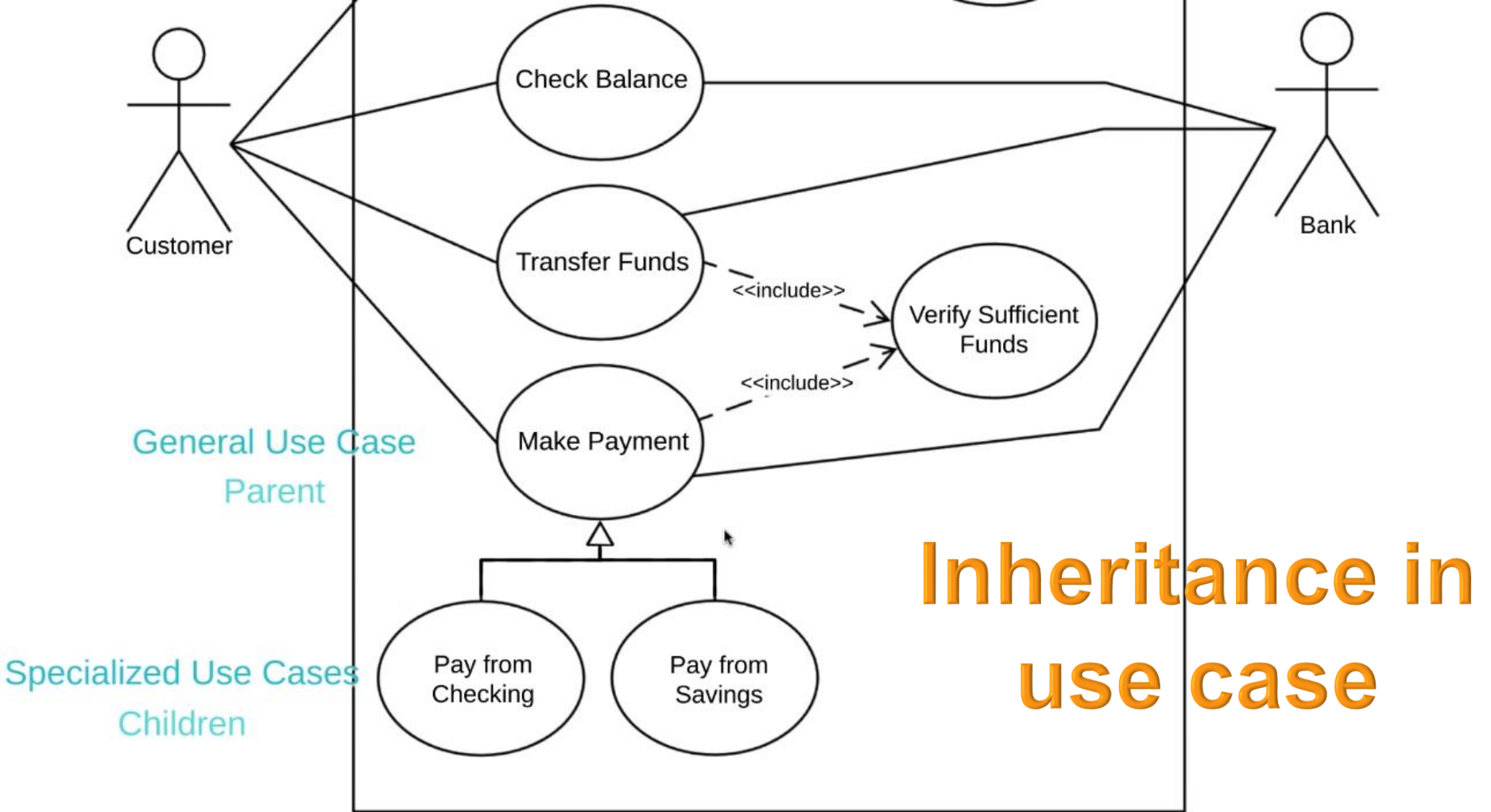
Include and extend relationship



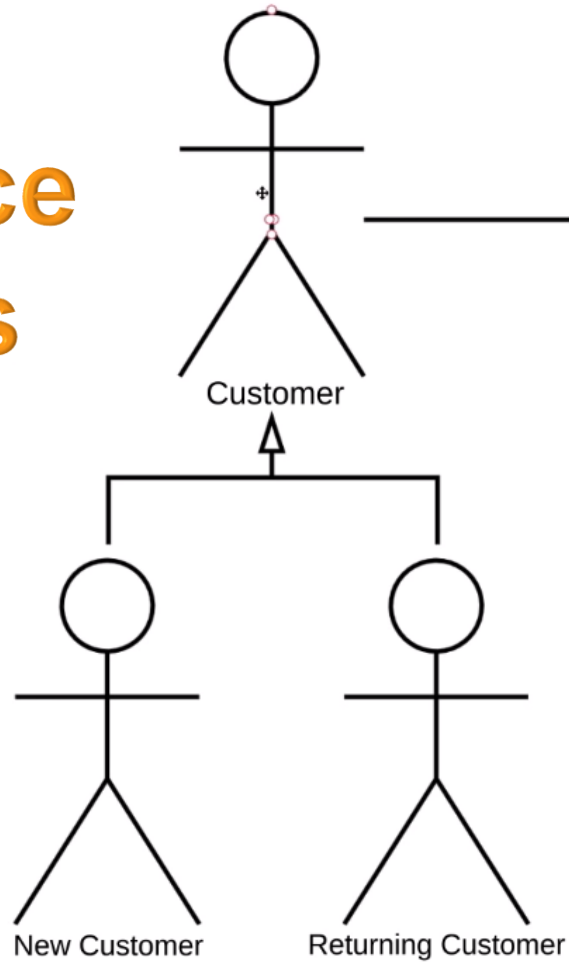


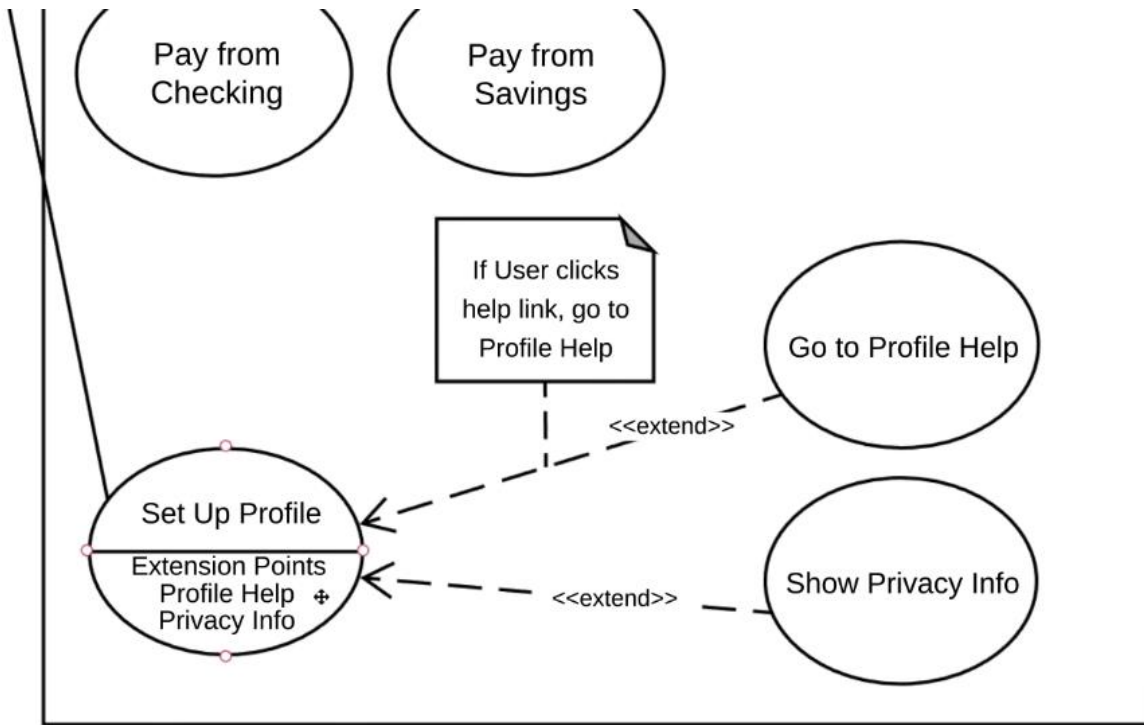


Include and
extend
relationship

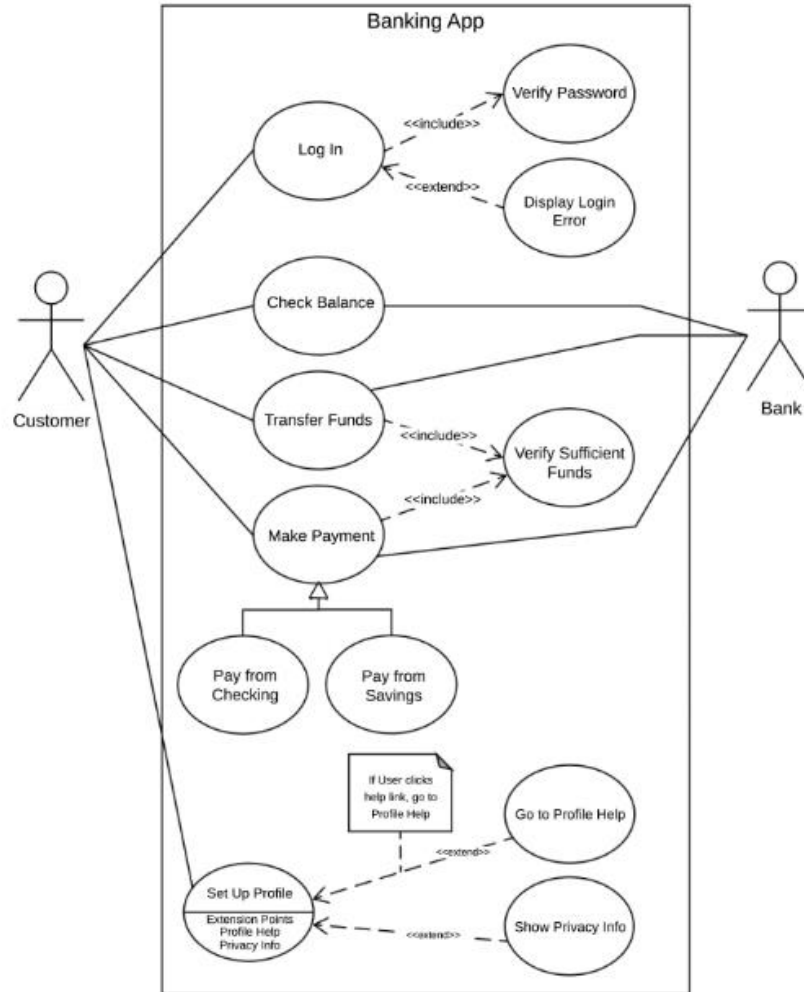


Inheritance in Actors





Extension Point

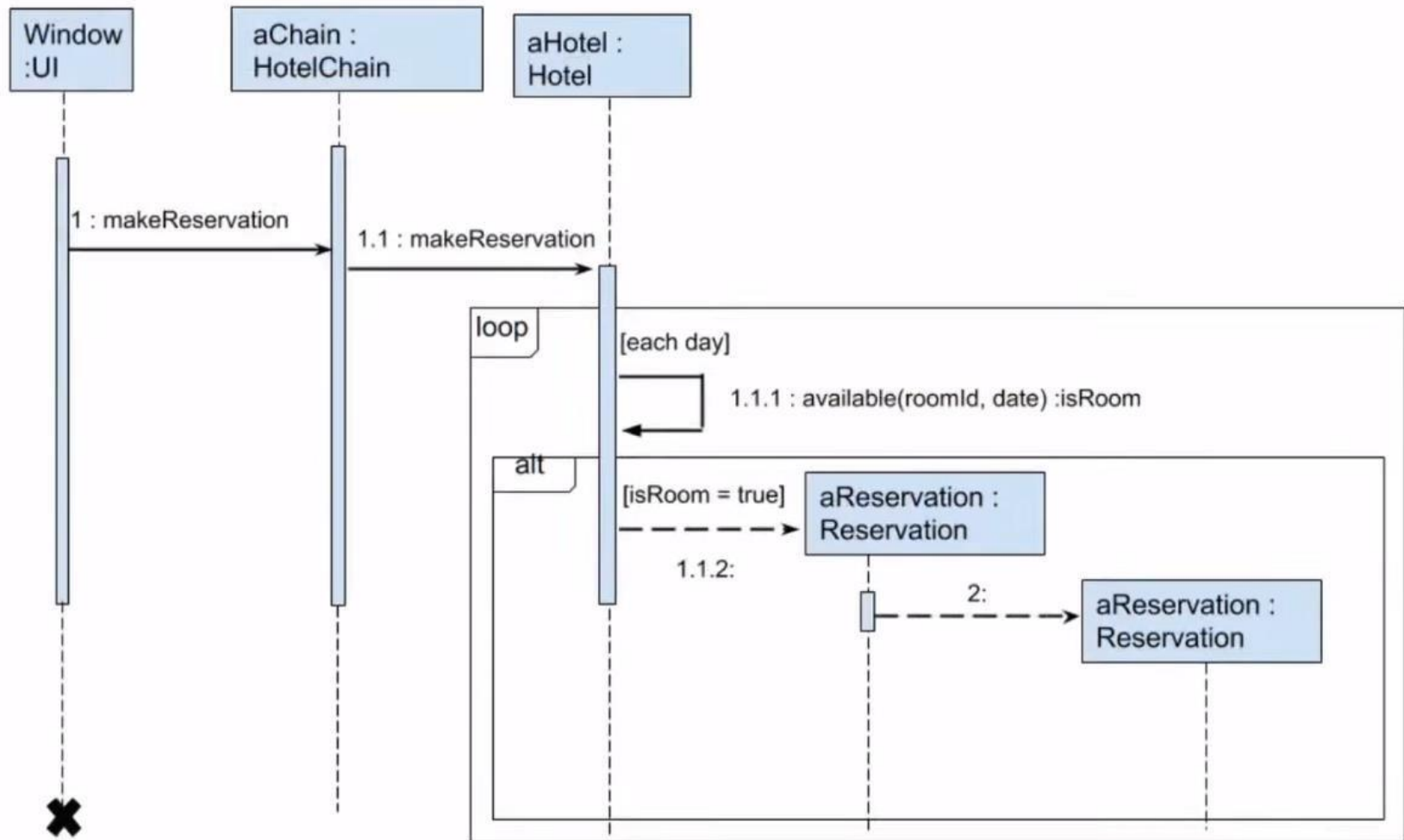


Interaction Diagrams - Interactions among the different elements in the model

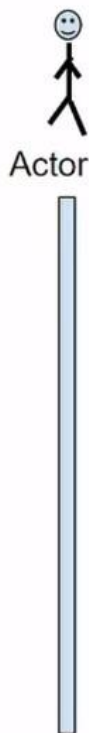
- **Sequence diagram**
- **Collaboration diagram**

Purpose of a Sequence Diagram

- To model high-level interaction among active objects within a system.
- To model interaction among objects inside a collaboration realizing a use case.
- It either models generic interactions or some certain instances of interaction.

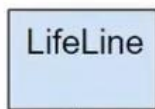


Actor



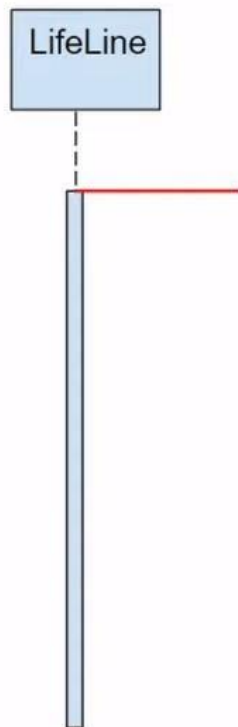
- A type of role played by the instance that interacts with the subject (for example, by exchanging signals and data).
- Actor represents the roles played by users, external equipment, or other actors.
- Actor does not necessarily represent a specific physical entity, but simply a specific role of some entity.
- A person can play the role of several different actors, and, conversely, an actor may be played by multiple different people.

LifeLine

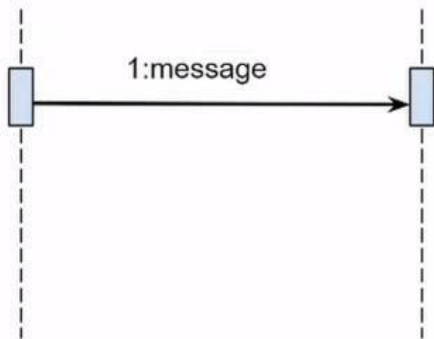


Represents an individual participant in the interaction and displays the passage of time.

Activation



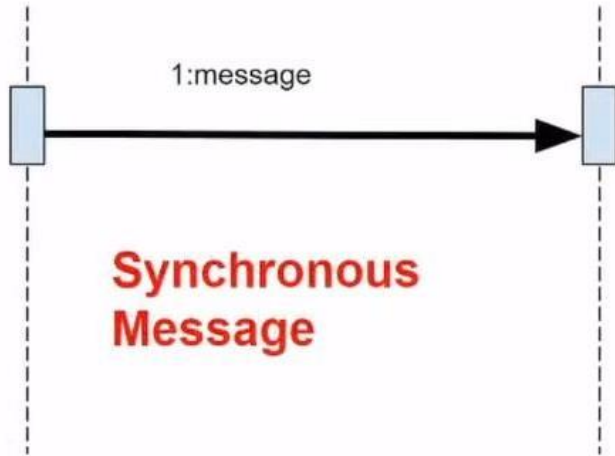
- The thin rectangle on the lifeline indicates the period during which the element performs an operation.
- Top and bottom of the rectangle are aligned with the start and end of a specific function.



Call Message

- Used to call procedures, execute operations, or designate individual nested control flows.
- One end of the arrow always touches the control focus or lifeline of the client object that triggers the message.
- The tip of the arrow points at the lifeline of the object that receives the message and takes action as a result. At the same time, this object also often receives a control focus, becoming active.

Call Message



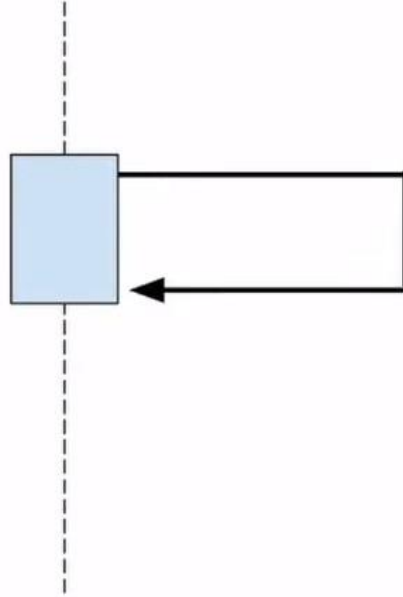
The message is synchronous (depicted with a filled arrow), when the client sends a message, for example, to the server and waits for a response before doing anything else.

Call Message



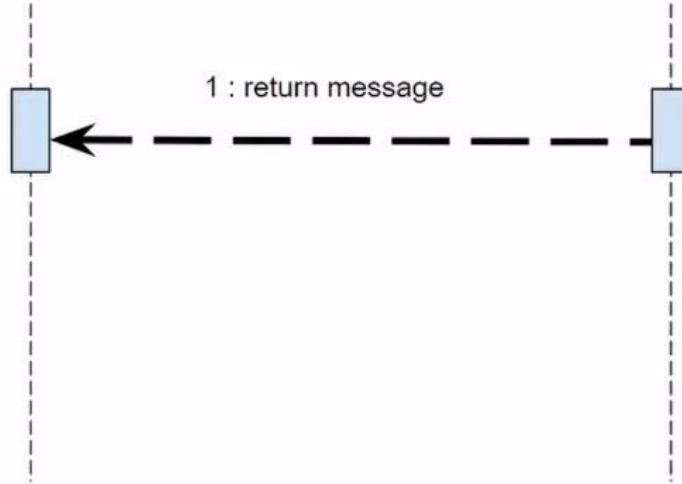
Message is asynchronous (depicted with a regular arrow), when the client continues to perform operations without waiting for a response.

Self Message



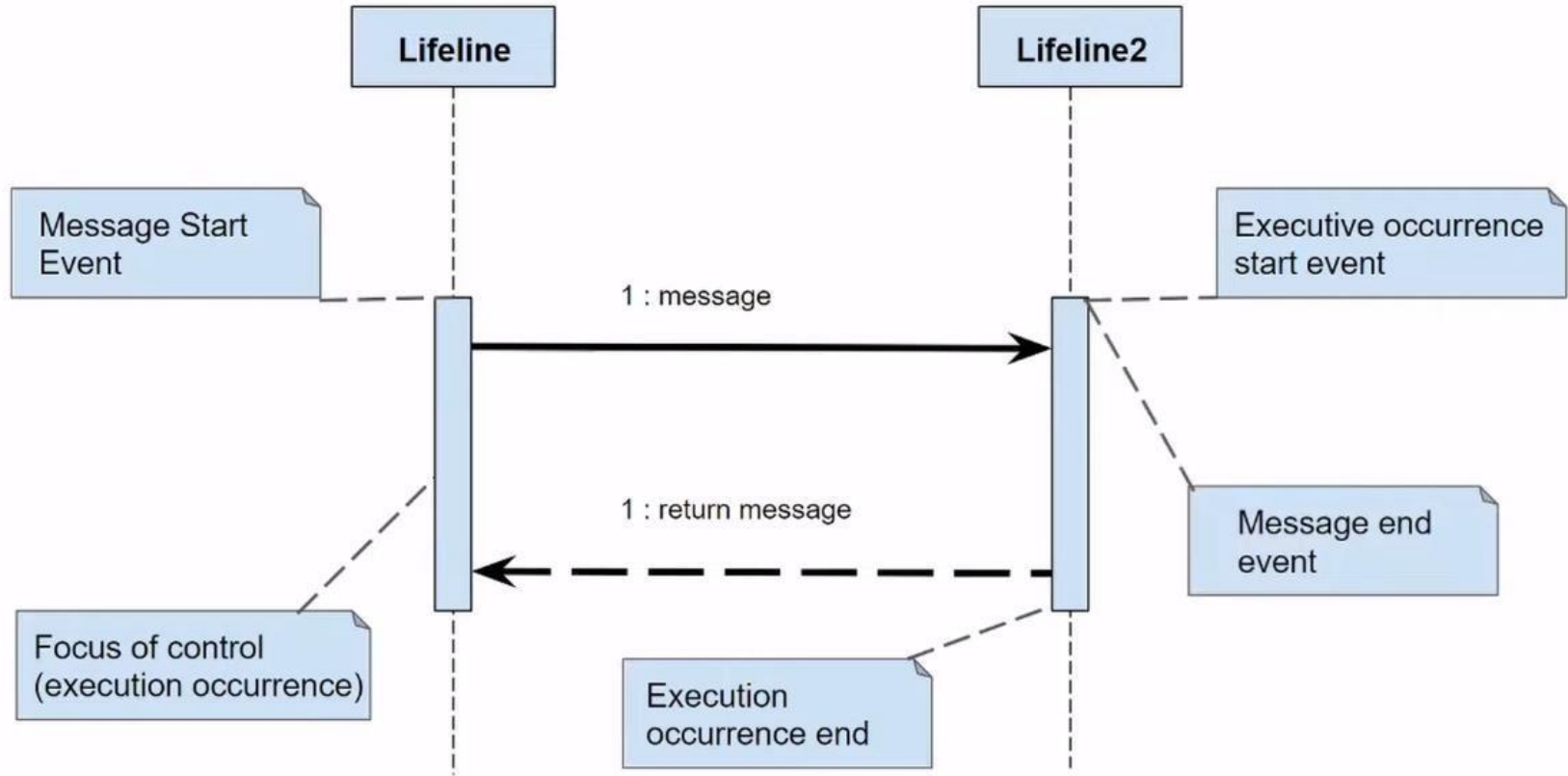
A participant sends a message (command) to itself.

Return Message

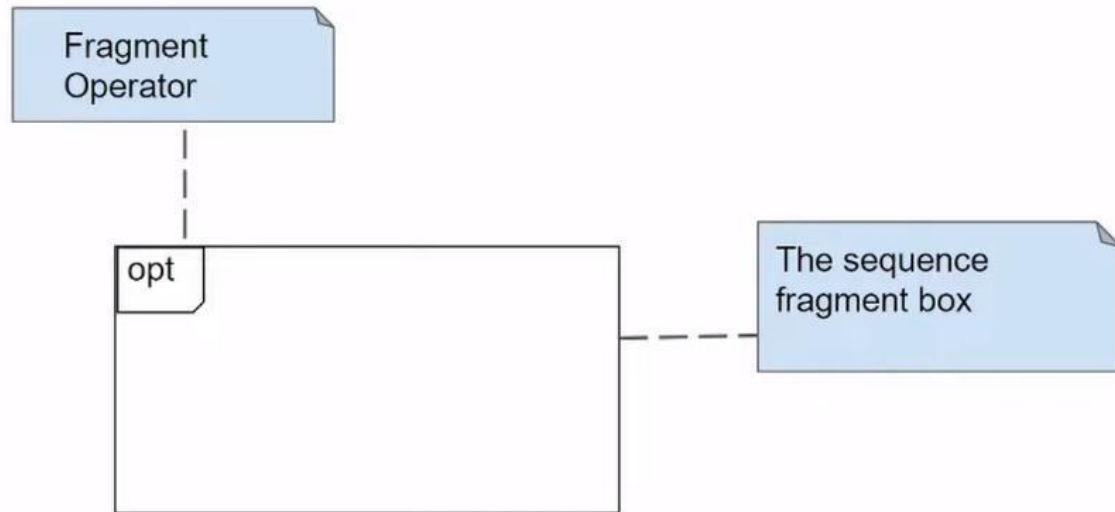


Return message identifies the specific communication between the lifelines of an interaction and represents the transfer of information to the caller of the corresponding message.

Focus of Control



Sequence Fragments



alt

Multiple alternative fragments, only the one whose condition is true is executed. Messages (group of messages) are separated from each other by horizontal dashed lines. This type is used to simulate conditional statements (if-then-else) and select statements (case or switch)

loop

Cyclic message processing.
Used to simulate loops

opt

The fragment is executed only if the provided condition is true and it is a call to an additional message (group of messages) under some condition.

It is similar to the “alt” fragment when the shorthand conditional statement (if-then) is used.

seq

Non-strict sequential message processing.

Messages are separated from each other by horizontal dashed lines and can be processed in any order except for messages received by a single object.

